

Introduction to Deep Learning

Convolutional Neural Networks:- (CNN)

* What if we want to identify if a pattern $\begin{bmatrix} 3 \\ 6 \\ 2 \\ 1 \\ 7 \end{bmatrix}$ is present in a 128×1 vector? (in any position).

→ To build a FFNN type model for this task, we may need extremely humungous amounts of data.

* How can we build a model that can be trained with a limited dataset?

→ We need to change the architecture. (CNN).

→ CNN makes the neural networks, shift invariant.

$$\rightarrow \left[\dots 7 \ 2 \ 3 \dots \right] \approx \left[\dots 7 \ 2 \ 3 \dots \right]$$

Vector A

Vector B

$\gg$

shift invariance

$$\left[\dots \dots 7 \ 2 \ 3 \right]$$

Vector C

→ How to detect this pattern in the input?

- Do convolution operation

$$\left[x_1 \ x_2 \ x_3 \ \dots \ x_9 \ x_{10} \right]$$

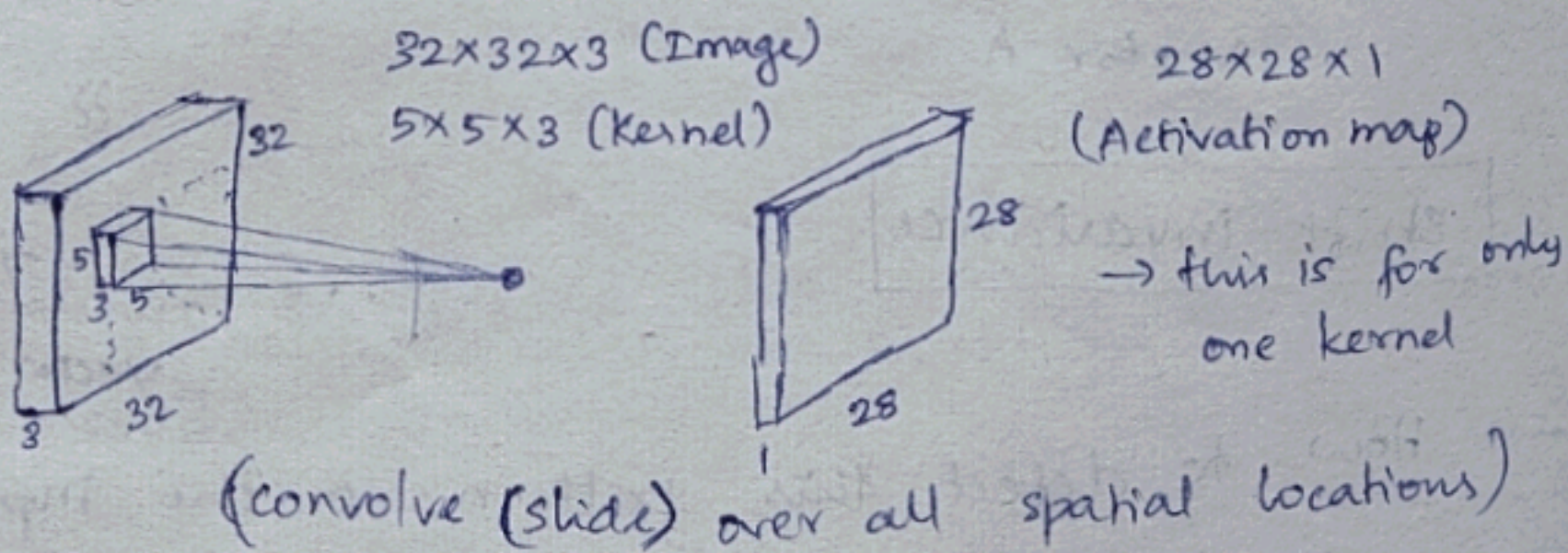
$[t_1 \ t_2 \ t_3] \Rightarrow$ traverse through this vector with one positional shift.

* The mathematical expression of convolution operation is written as,

$$y[k] = \sum_{n=-\infty}^{\infty} x[n] f[k-n]$$

* Through convolution operation, what we have effectively done is we segmented the data. The segmentation size is dependent on the size of the filter (kernel). Each segment is multiplied by a filter, which maps the segment to a scalar value.

* Filtering captures a pattern. By projecting input onto a filter (matched filter), a pattern is searched. Multiple such filters, ^(different) can be used, resulting in multiple channels. (all will be uncorrelated).



⇒ If we have 6 kernels, then activation map will be of size (28x28x6).

* Example Kernels:-

$$\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

→ Blur

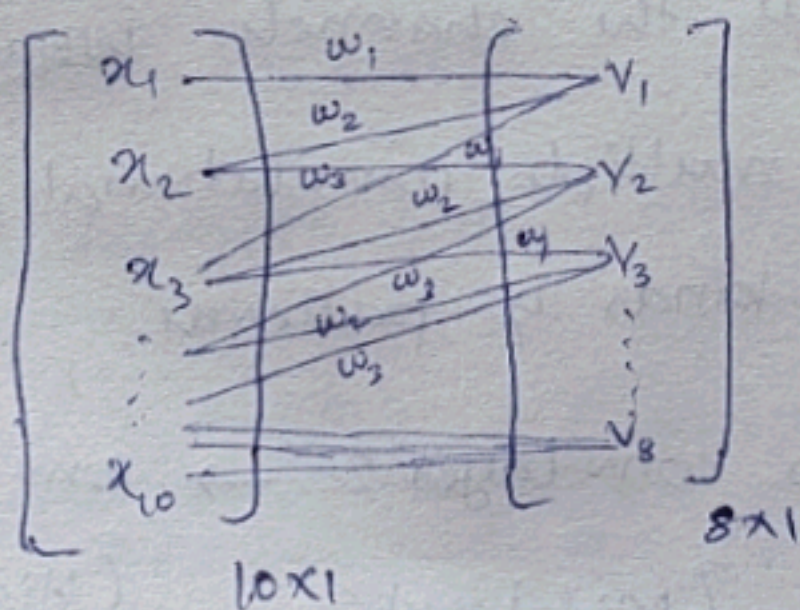
$$\begin{bmatrix} 1 & 1 & 1 \\ 1 & -8 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

→ Edge detection
(differential operation)

$$(I * K)_{ij} = \sum_{a=0}^{m-1} \sum_{b=0}^{n-1} I_{i-a, j-b} K_{a,b}$$

* How do we learn these kernels during training?

$$f = \begin{bmatrix} w_1 \\ w_2 \\ w_3 \end{bmatrix}$$



This almost looks like a FFNN but the thing is

it is not fully connected, the no. of connections to a

neuron in the next layer is same as the size of the

kernel. Also, there exists parameter sharing. Though

we have more neurons in the next layer, the no. of

parameters we have here is $(3 \times k)$ which is very less.

* The weights are sparse and shared.

* For an input of size $W \times H \times D$, and

For a filter/kernel of size $F \times F \times D$

→ Output per kernel is, $(W-F+1) \times (H-F+1)$

→ Total output for K kernels is,

$$(W-F+1) \times (H-F+1) \times K$$

* This first level of convolution filters the selected no. of patterns from the arbitrary set of patterns in the input, across all the channels. What next?

- Do nesting of multiple convolutional layers, which looks for different kinds of patterns.

Conv Layer 1 → Conv Layer 2 → Conv Layer 3
(Low Level Features) (Mid Level Features) (High Level Features)

Subsequent layers, patterns over patterns.

Receptive field increases with depth.

* There are a lot of data (patterns/outputs of one conv. layer) which gets generated, can we downsample it?

- Striding and Pooling

Padding:-

Padding borders of input to obtain feature info. from corners. Padding length P depends on F . $\{P = \frac{F}{2}\}$

Striding:-

Filtering is not continuous, while convolving, a certain sequence of datapoints are skipped. Eg. First, ^{datapoint in first} column is considered for convolution and the next datapoint is not from the second column, it is skipped and the ~~the~~ datapoint from third column is considered. Stride length S is the downsampling factor.

The output feature size becomes,

$$= \left(\frac{W - F + 2P}{S + 1} \right) \times \left(\frac{H - F + 2P}{S + 1} \right) \times K$$

Pooling:-

Take a bunch of pixels together and pool them into one value. This reduces feature size even further.

* Types of Pooling:- (i) Max pooling
(ii) Average pooling. Pooling itself is a separate layer.

* Pooling handles "pixel-jitter".

* Pooling can be seen as a separate convolution operation.

* There is no weight learning happens in Pooling layer.

* Pooling helps in preventing pixel-fitter by doing local invariance, globally equivariant.

* Striding and Pooling also helps in increased Receptive Field.

* Pooling helps in attaining shift invariance much quicker.

* Average pooling:-

$$\frac{1}{m \times n} \sum_{i=1}^m \sum_{j=1}^n x_{ij} \rightarrow \text{Differentiable}$$

Max Pooling:-

$$\max(x_{11}, x_{12}, \dots, x_{mn}) \rightarrow \text{Not differentiable}$$

$$\sum w_{ij} x_{ij} \rightarrow \text{Differentiable}$$

(keeping all weights 0,
except the max value weight,
that will be 1)

Activation Function:- "Softmax" at the output layer of

~~FC~~ FC FFNN (MLP) at the last layers.

Different CNN Architectures :-

AlexNet (62 M parameters) :- (2012)

- * Early Layers - Larger Kernels and strides (11×11 - Ker) ^{then 5×5}
4 - stride
- * Later Layers - Smaller Kernels and strides (3×3 - Ker)
2, 1 - stride
(Receptive field - almost whole image)
- * ReLU, SGD with momentum, max pooling, dropout
- * Data augmentation - Flipping, Jittering, Cropping, Color normalization

* Limitations:-

- Not deep enough
- Too big kernels (Lacks identification of finer patterns)
- No proper normalization
- Too many parameters (predominantly came from FCFFNN layers, the contribution from Conv. layers towards the parameter count is much lesser).

VGG Net (138 M parameters) :- (2014)

- * Smaller filter size, stride - 2
- * More depth (13 conv. layers with 3×3 filter size)
- * Receptive field grows gradually, captures more patterns
- * Limitations - More depth $\rightarrow$ More vanishing gradients

Google Net (6.8 M parameters) :- (2015)

* Instead of using a single kernel in every convolutional layer, this uses multiple kernels simultaneously.

($1 \times 1, 3 \times 3, 5 \times 5$ - simultaneously)
* Each kernel has a different receptive field. Then, concatenate the features of all kernels - inception.

* 1×1 - weighted summation across channels.

(Dimensionality reduction - removing channels).

* Bottleneck Layer - Global pooling (reducing parameters)

* Limitation :-

- Cannot go deeper - needed for better generalization

* To overcome vanishing gradients, the loss function is defined like this,

$$\text{Total Loss} = 0.31 L_{D_1} + 0.3 L_{D_2} + L_{\text{End}}$$

- It gives multiple outputs at different depths,

- only final output is retained in testing.

Res Net-18 (12M parameters) :- (2016)

* Introduction of skip connections (more depth)

* Batch Normalization

* Bottleneck Layer (only important features are passed through)

* Dense net (Feature Reuse) (2016) - CSV 001938

Instead of skip connections (where the outputs are meant to be added with the previous layer outputs), the outputs are concatenated to have an expanded receptive field.

* Res Next (Inception + ResNet)

* Neural ODE

Inspired by ResNet, each layer is one order of differential equations

UNet (31M parameters) :- (2015)

* Developed for Image segmentation (distinguishes foreground from background)

* Autoencoder architecture :- Results in denoising (regression), segmentation (classification)

* Encoder (compressor) - Extract "principal" features and reduces size.
↳ (semantic features)

* Decoder (expander) - Reconstructs "principal" features

* Skip connections across layers of same size.

→ Provides feature-location information, compensates information loss.

DeepLab V3+ :- (2018)

* Morphed version of Google Net.

* Encoder :-

→ Atrous convolution - alternative pixel convolution, results in larger receptive field

→ Atrous spatial pyramid pooling

→ Inception + Pooling

* Decoder :-

→ Similar to U-Net

→ Low resolution features (semantic info)

→ Upsampled features (spatial info)

Latest Models :-

* Vision Transformer (ViT - 2023)

- Converts image into timeseries and then uses transformers for image classification.

* YOLO 26 (2026)

- Encoder (Backbone), Neck, Decoder (Head)

- SOTA for Image classification

GradCAM :-

* Tells us which spatial feature region caused the decision.

* Gradient of the output logit w.r.t. features
* A_{kij} - Feature at k^{th} channel and (i,j) spatial position.

→ Computed from backprop Gradient $\frac{\partial y^c}{\partial A_{kij}}$

→ Global ^{average} pooling - neuron importance weight:-

$$a_k^c = \frac{1}{L} \sum_{ij} \frac{\partial y^c}{\partial A_{kij}}$$

$$\rightarrow \text{Grad-CAM} = \text{ReLU} \left(\sum_k a_k^c \cdot A_k \right)$$

→ Positive gradient implies affirmative contribution by (i,j) pixel to class c .

Recurrent Neural Networks :- (RNN)

* Both FCFFNN and CNN require data of fixed length.

* How to handle data of unknown length / varying length?

* What if a timeseries data is of correlated inputs?

- The output y at t^{th} instance is a function of the input at t^{th} instance and also the inputs before the t^{th} instance, i.e., $t-1^{th}$ instance.

$$y_t = f(x_t, x_{t-1})$$

→ RNN handles this

* We need some assumptions on input-output correlation

$$y_t = f(x_{-\infty} \dots t \dots \infty)$$

→ Finite Length Response:-

$$y_t = f(x_{t-N} \dots t)$$

$$= f(x_t, x_{t-1}, x_{t-2}, \dots, x_{t-N})$$

→ Infinite Length Response:-

$$y_t = f(x_{-\infty} \dots t)$$

$$= f(x_{t-N} \dots t, y_{t-N} \dots t)$$

* Assumed to be a "Causal" system

* Before RNN came into existence, there were

→ Autoregressive - AR(N) process

$$y_t = \sum_{i=1}^N W_i y_{t-i} + x_t$$

→ Moving Average - MA(N) process

$$y_t = \sum_{i=0}^N W_i x_{t-i}$$

→ ARMA:- $y_t = \sum_{i=0}^N W_i x_{t-i} + \sum_{i=1}^M V_i y_{t-i}$

→ Wold's representation theorem, - ARMA process is universal.

- Any WSS process is = deterministic + stochastic process

→ Non-Linear Autoregressive with exogenous input

$$(NARX) :- y_t = f(y_{0 \dots t-1}, x_{0 \dots t})$$

State Space Model:-

How do we address the memory problem? The simplest option is State Space Model.

→ Transform the input variable into a new variable which captures the relevant information from the past.

(hidden state)

Hidden state: $h(t) = g(x_t, h_{t-1})$

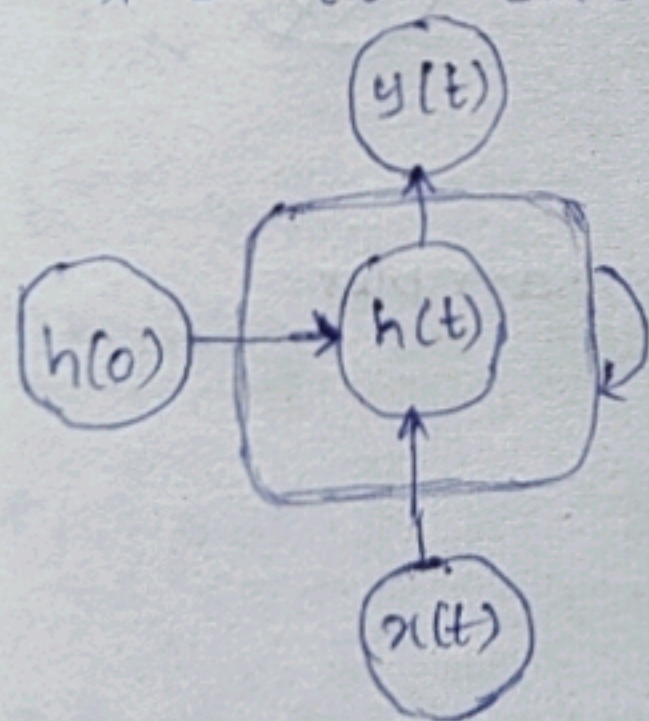
$$y_t = f(h_t)$$

$g(); h()$ are NN that can be learned.

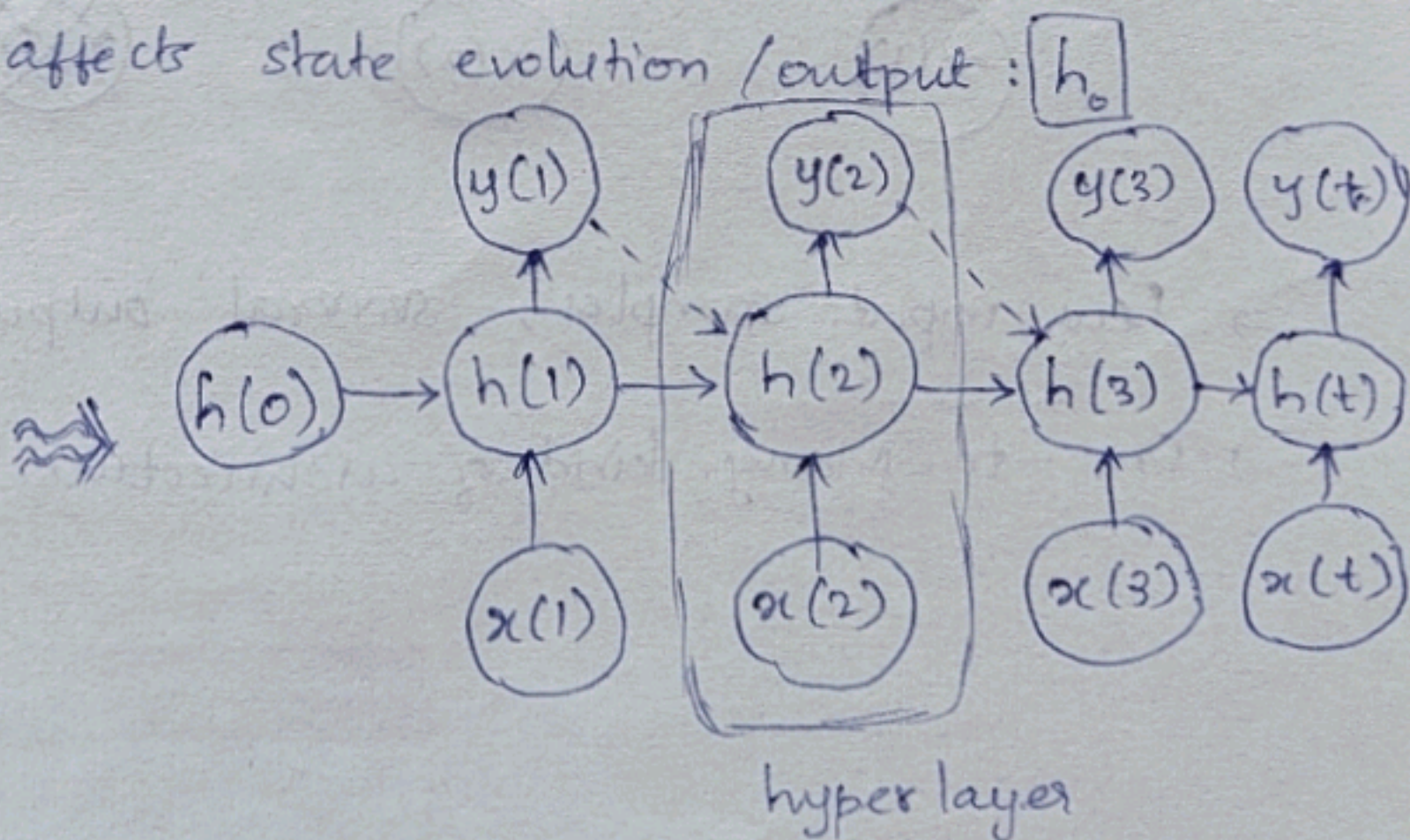
Parameters of $g(); h()$ are shared for all "t". (across layers)

* It summarizes / retains sufficient statistics / information from the past.

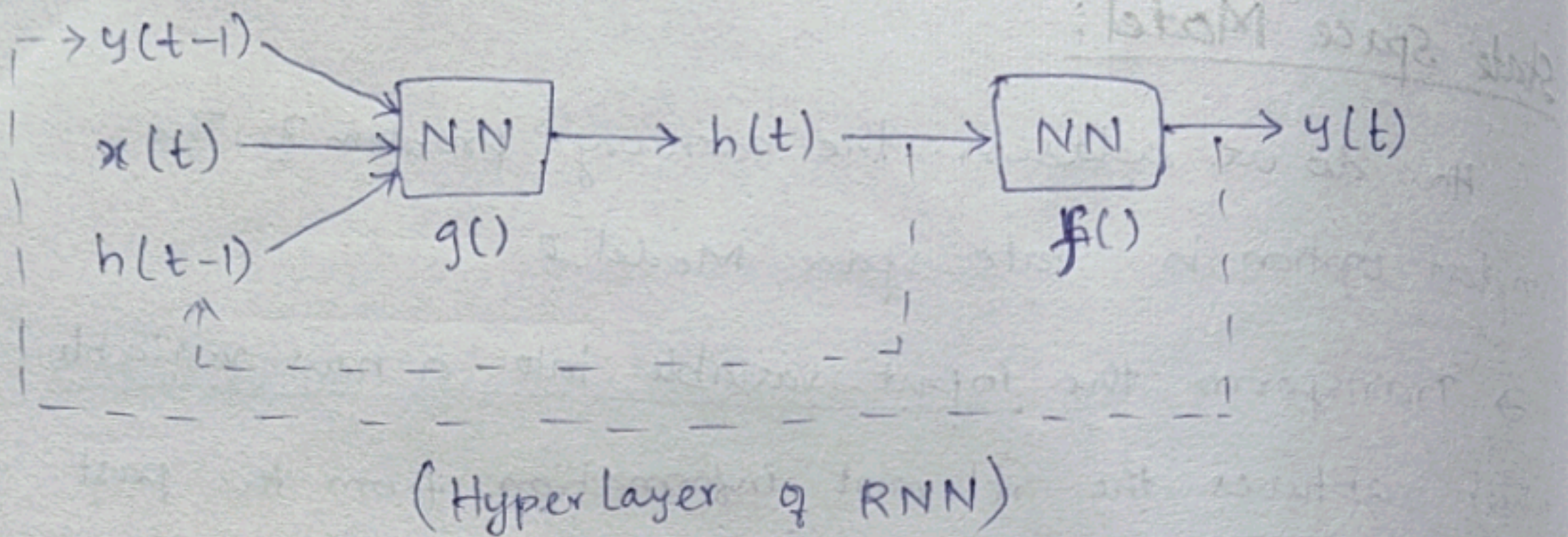
* Initial state affects state evolution / output: h_0



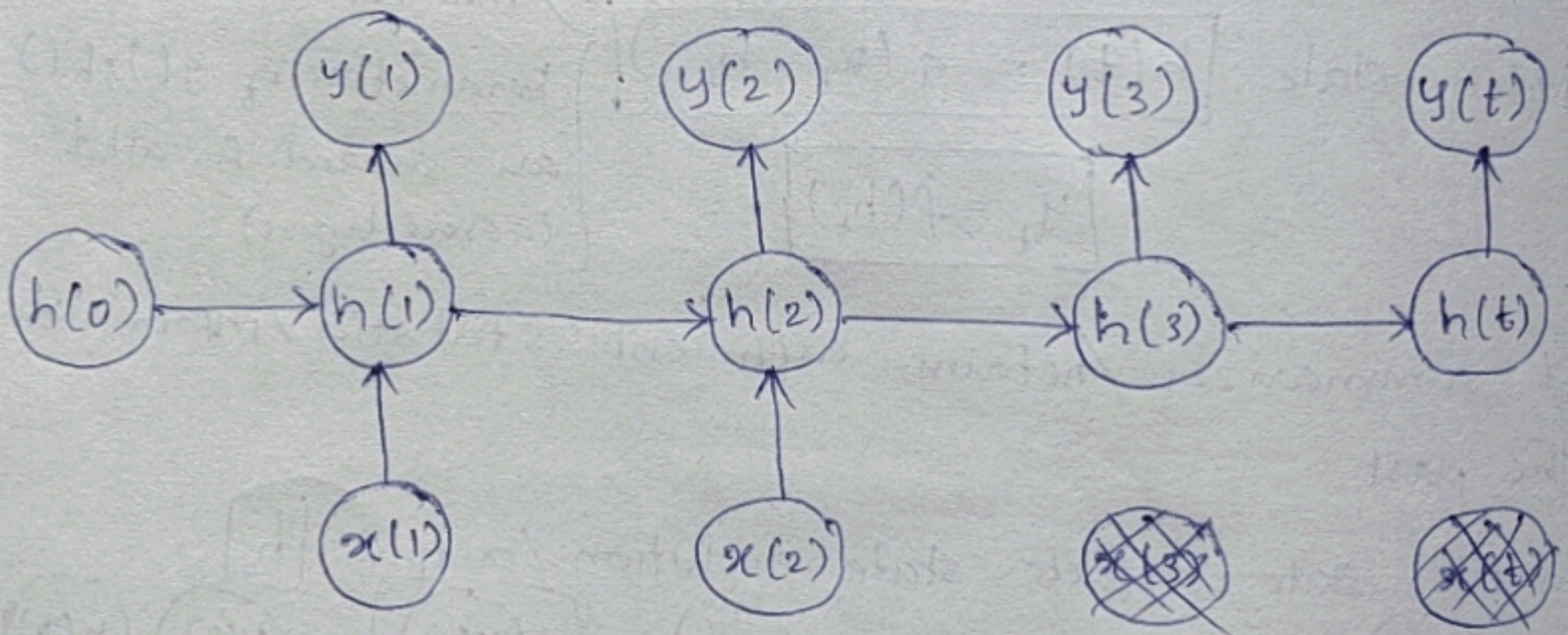
Typical RNN
Architecture



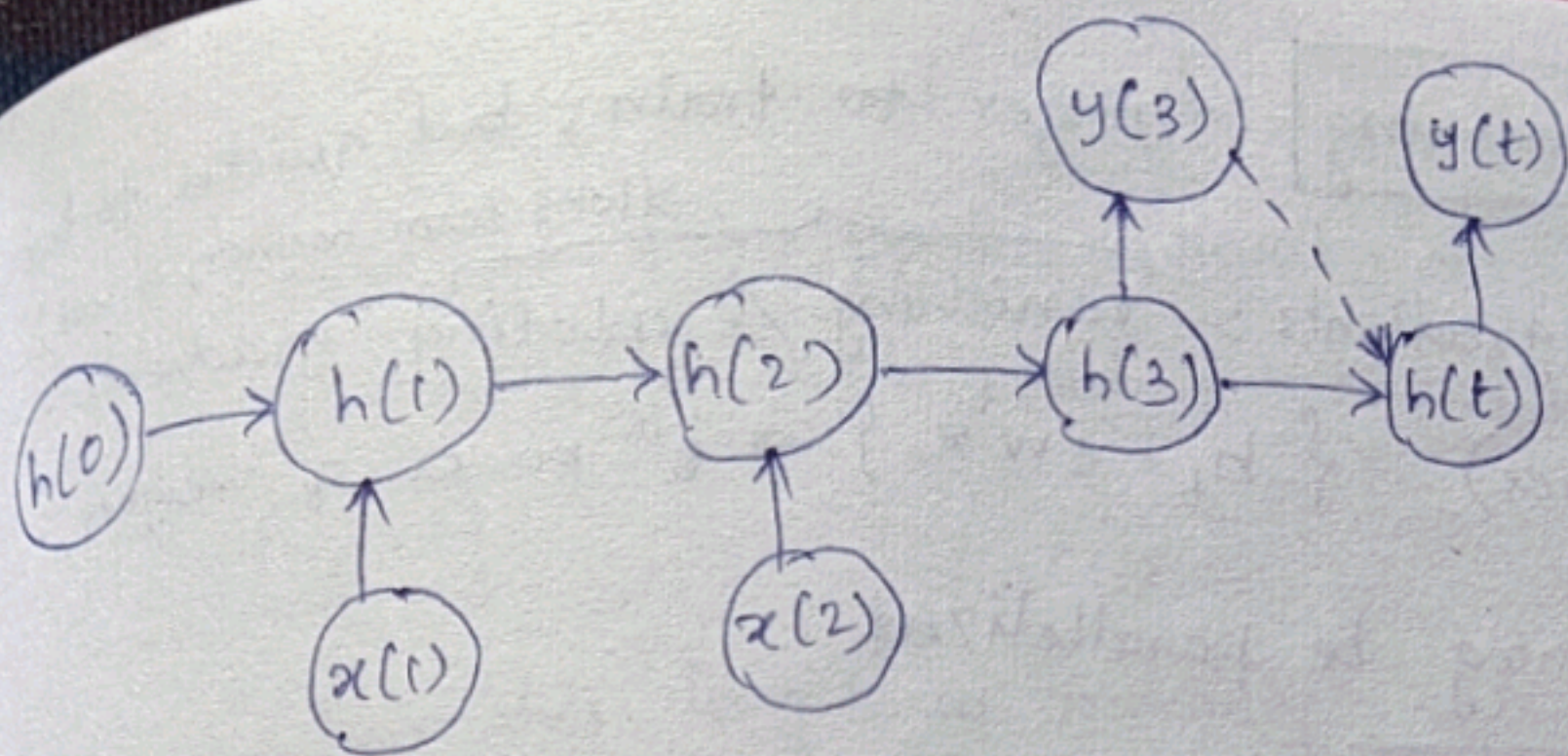
* The $g()$; $f()$ - NN becomes independent of time as the inputs and outputs are correlated w.r.t. time.



Different forms of RNN:-



- Few input samples, several output samples.
- One-to-Many kind of architecture.



→ wait till all the inputs are fed in and then start generating outputs.

Examples:-

- * Time series (stock market) prediction - One to one.
- * Sequence (next word) prediction, speech recognition, text classification, Sentiment analysis, weather forecast, anomaly detection - many to one.
- * Sequence to sequence conversion, language translation, data summarization, text to speech and vice versa - many to many.

Training an RNN:-

- * Unfolding an RNN over time gives an MLP
- * Backpropagation over time (over the unrolled NN)

- * Sequential training - longer to train; but quicker to test (long term memory is lost)
- * Issues with gradients: - vanishing / exploding gradients.
(stability issues) $\{h_t = c w^t x_0\} \rightarrow t^{\text{th}}$ power of weights.

* Can the training be parallelized?

- Teacher Forcing $\rightarrow$ During training, feed ground truth

y_{t-1} for computation of y_t . It works well for a certain kind of architectures, doesn't generalize well for all type of architectures. If h_{t-1} to h_t connection doesn't exist, training can be parallel and faster.

Gating to Retain Memory:-

* Should the previous state always be used?

- Skipping through time is equivalent to adding a previous state gated with a counter.

* Gating the previous state: $\tilde{h}_t = g(x_t, r_t \cdot h_{t-1})$

- In computational graph, gradients are blocked when reset gate $r_t = 0$

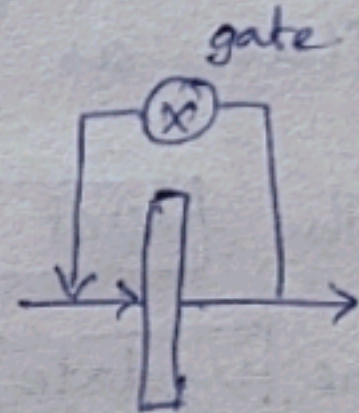
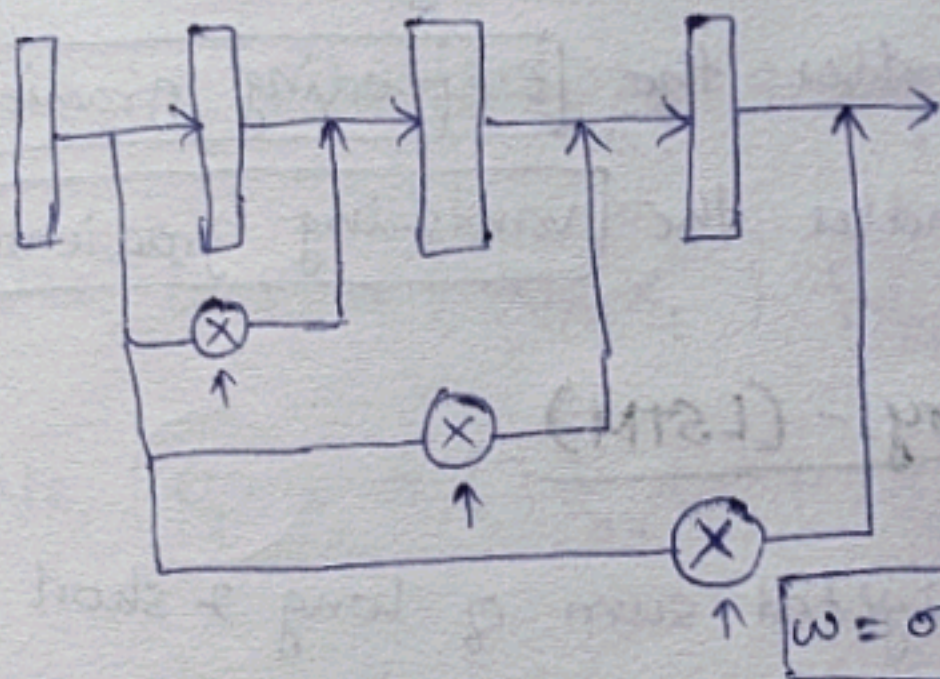
- It promotes short term memory; resets long term memory to nil - forget gate

* Gating at the output: $h_t = u_t h_{t-1} + (1 - u_t) \cdot \tilde{h}_t$

- In computational graphs, gradients flow directly from past (without an added product of derivative of additional function) when update gate

$$u_t = 1$$

- Promotes long term memory; retains previous state as current.



Gated Recurrent Unit (GRU): -

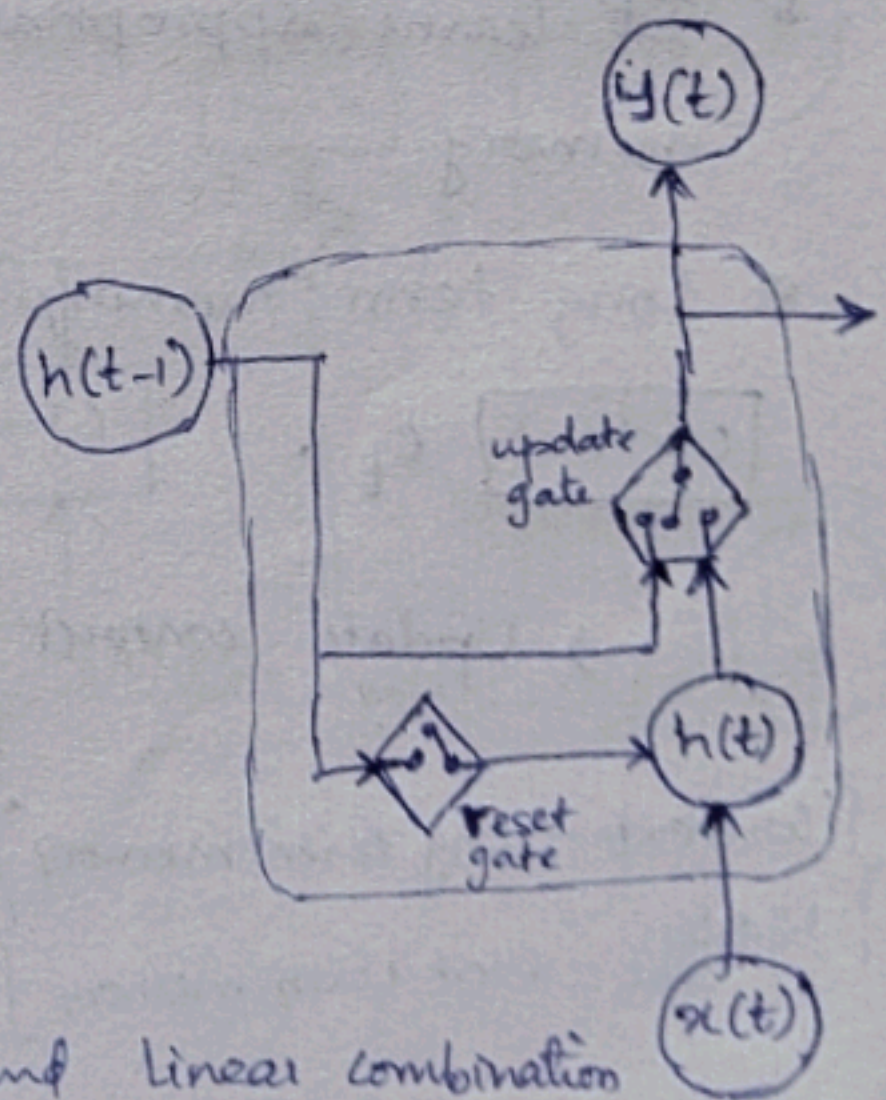
$$r_t = \sigma(W_{rx}x_t + W_{rh}h_{t-1} + b_r)$$

$$u_t = \sigma(W_{ux}x_t + W_{uh}h_{t-1} + b_u)$$

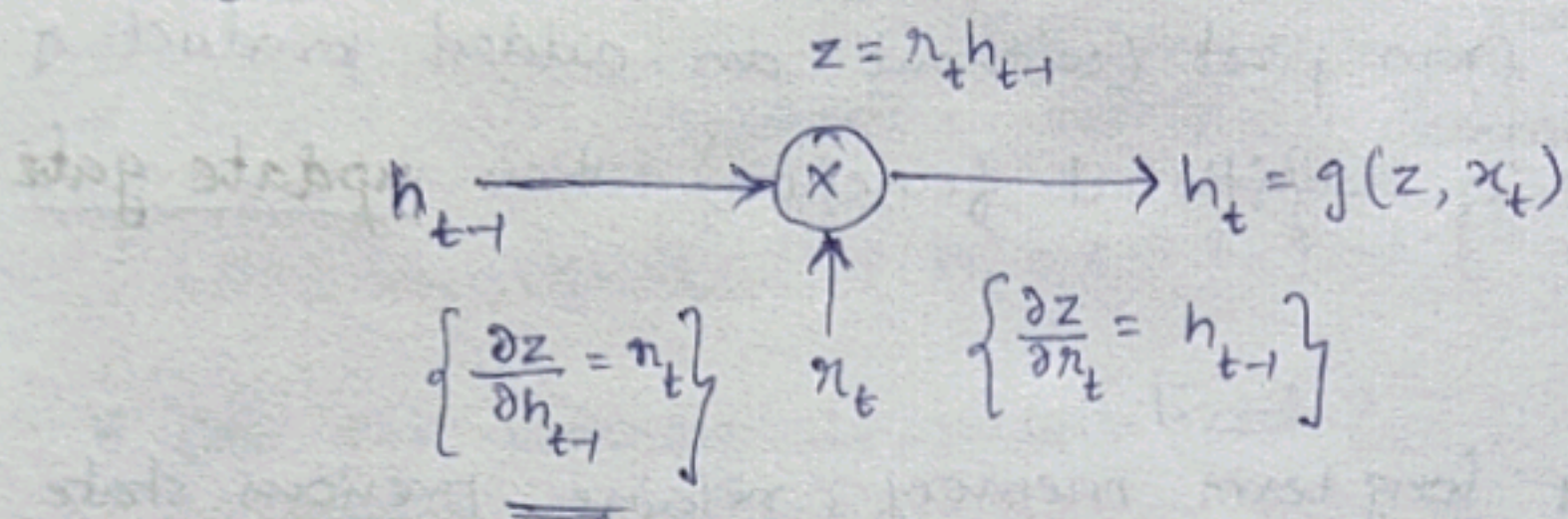
if, $r_t = 1$ and $u_t = 0$

then it is same as RNN

σ (sigmoid) brings differentiability and linear combination



Why this works?



So, the gradient flow stops, (interrupted) or it allows the flow as per the n_t .

→ The reset gate handles the exploding gradients problem.

→ The update gate handles the vanishing gradients problem.

Long-Short Term Memory:- (LSTM)

* GRU's update :- weighted sum of long & short term states, weights are close to 0 or 1.

* ^{LSTM} ~~It~~ learns appropriate function of long & short term memory.

* Long term memory is retained through cell memory (or) context c_t .

→ Update context $c_t = f_t \cdot c_{t-1} + i_t \cdot g_c(x_t, h_{t-1})$

$\left\{ \begin{array}{l} \text{context - long term memory} \\ \text{hidden state - short term memory} \end{array} \right\}$

→ Forget or retain with a gate f_t
 → Usefulness of current input decided with input gate i_t

* Short term memory is computed from the context itself.

→ Update current state $h_t = o_t \cdot g_h(c_t)$

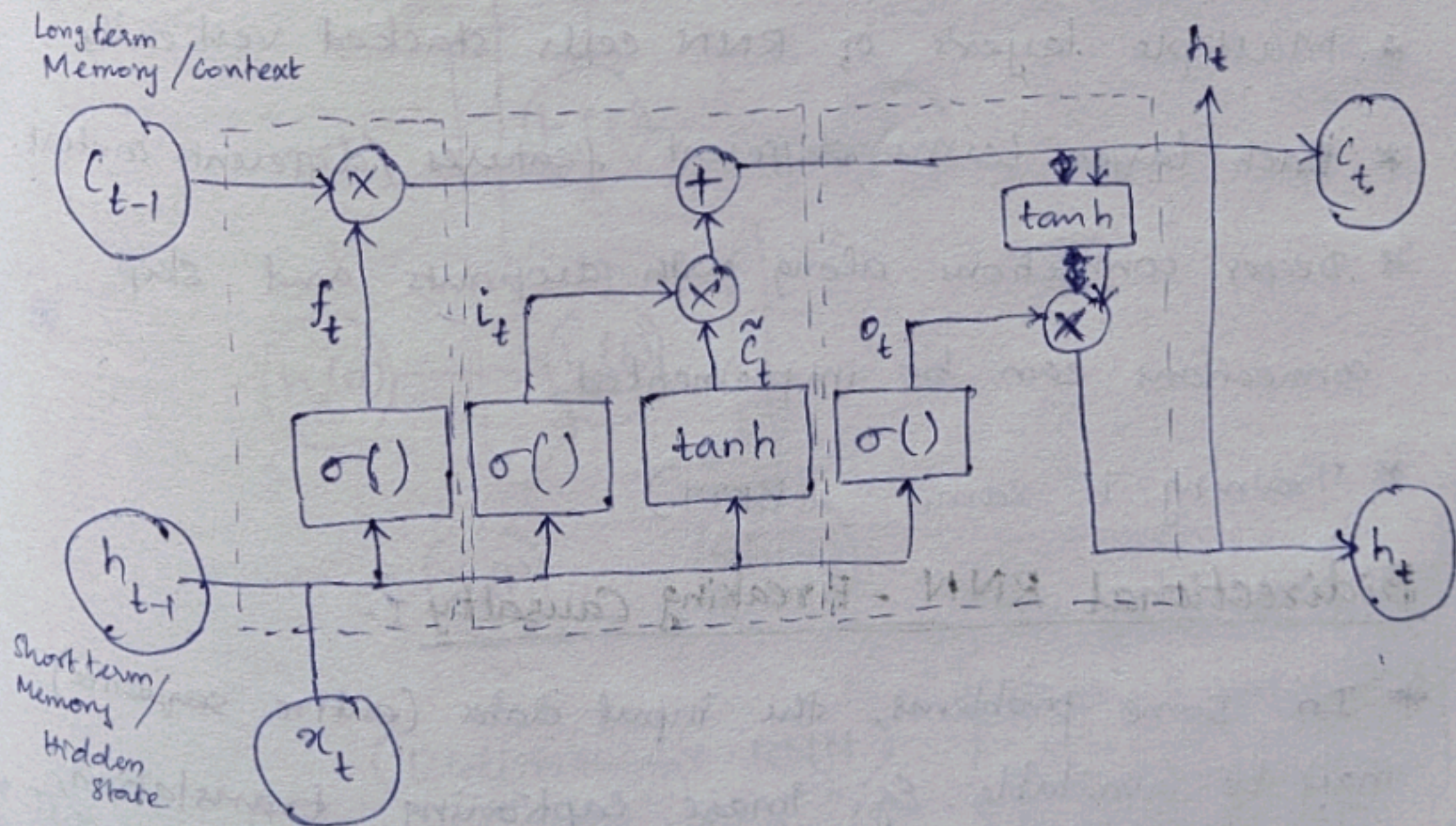
- Output gate decides if this state is relevant to be carried forward.

* Output is computed by $y_t = f(h_t)$

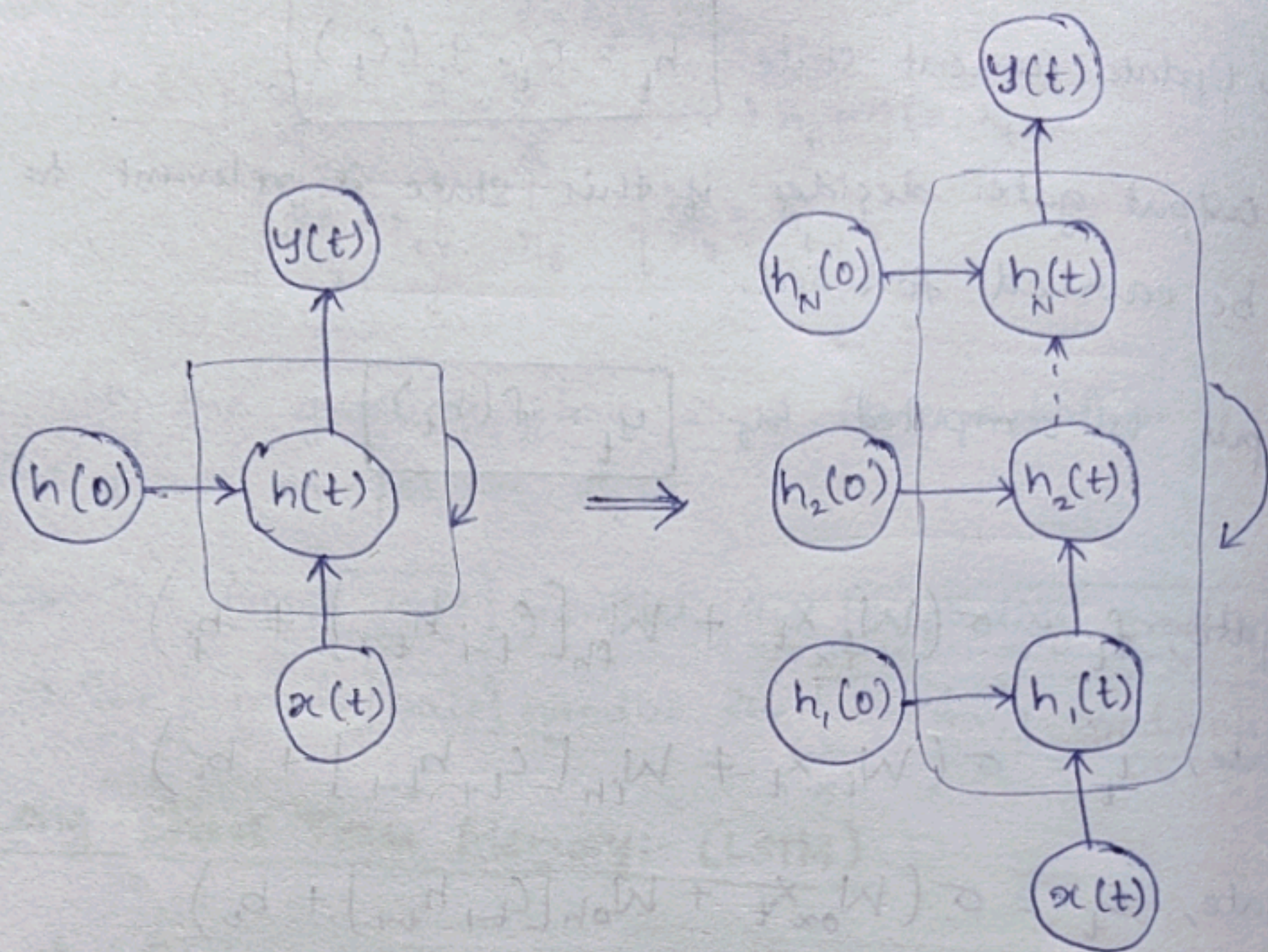
Forget gate, $f_t = \sigma(W_{fx}x_t + W_{fh}[c_{t-1} h_{t-1}] + b_f)$

Input gate, $i_t = \sigma(W_{ix}x_t + W_{ih}[c_{t-1} h_{t-1}] + b_i)$

Output gate, $o_t = \sigma(W_{ox}x_t + W_{oh}[c_{t-1} h_{t-1}] + b_o)$



Stacking RNN cells:-



- * Multiple layers of RNN cells stacked vertically
- * Each layer learns different features / different context.
- * Deeper connections along with dropouts and skip connections can be implemented.
- * Training is same (BPTT).

Bidirectional RNN - Breaking Causality:-

- * In some problems, the input data (entire sequence) may be available. Eg:- image captioning, translation, summarization, etc:-. And the output length may

still be unknown or input-dependent.

* Output ~~from~~ at time t may depend on inputs from past and future - captures more information.

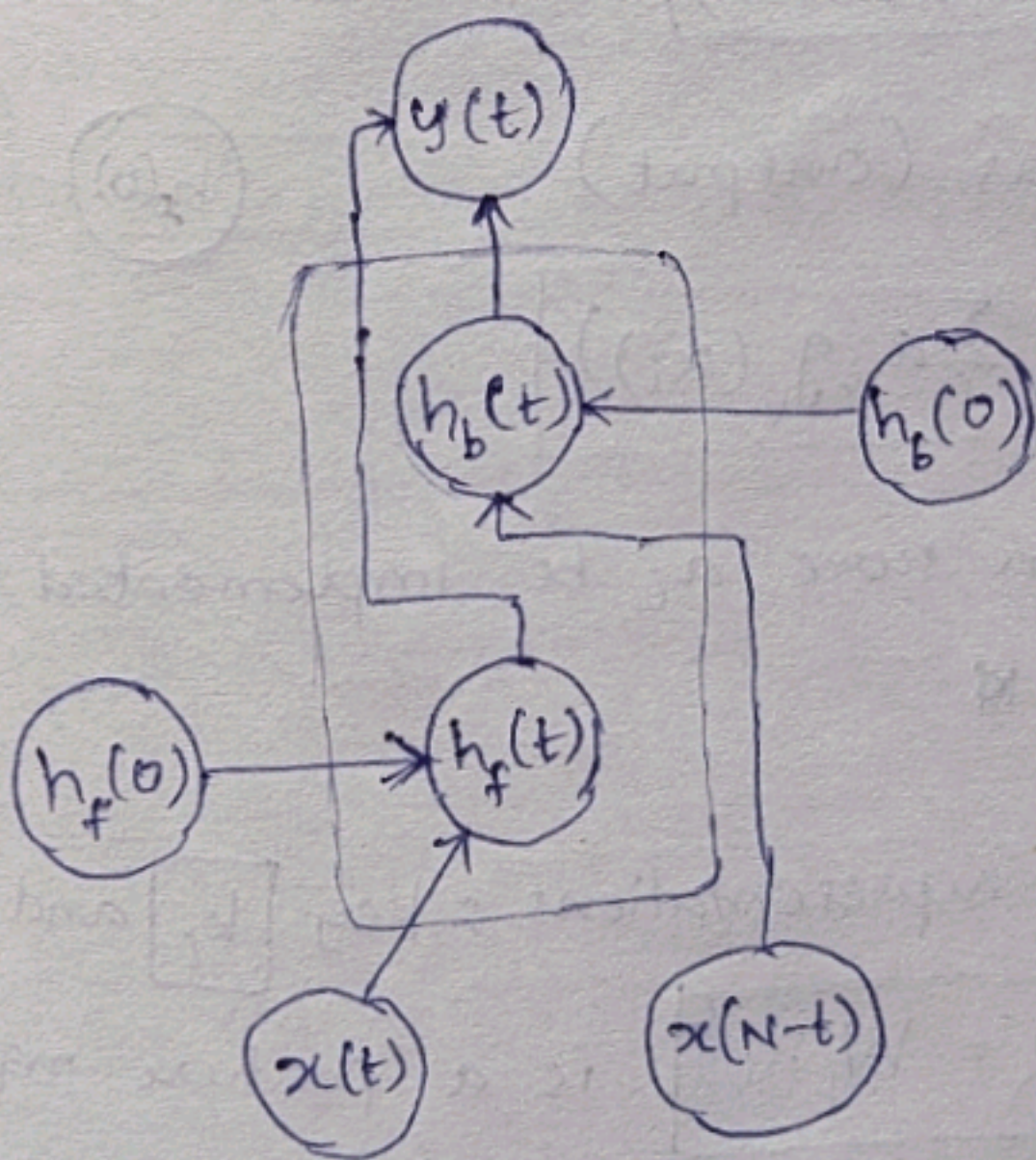
* So, instead of only one RNN unit, there are two RNN

state units: $g_f()$ and $g_b()$ - forward & backward:

Forward state :- h_{ft} :- inputs in forward sequence $t=1, 2, \dots, N$

Backward state :- h_{bt} :- inputs in backward sequence $t=N, N-1, \dots, 2, 1$

* No weights shared b/w forward & backward.



(Bidirectional RNN)

* Two unrolled NN are trained independently.

{ The same way,
Bidirectional LSTM
also exists and
Bidirectional GRU
also exists }

Attention Mechanism:-

* If the whole sequence is available, can we access it at all times?

$$y_t = f(x_1, \dots, x_t) \text{ instead of } y_t = f(h_{ft}, h_{bt})$$

* For a current output/state, how relevant is a sample in the input state sequence?

* We need to compute an attention score which will tell how relevant is, the samples in the input sequence.

$$\text{Attention, } a_i = s(h_{t-1}, x_i)$$

Update the state as, (output)

$$h_t = g(h_{t-1}, \sum_i a_i g_v(x_i))$$

* Can the attention score a_i be implemented with an MLP? ~~is~~

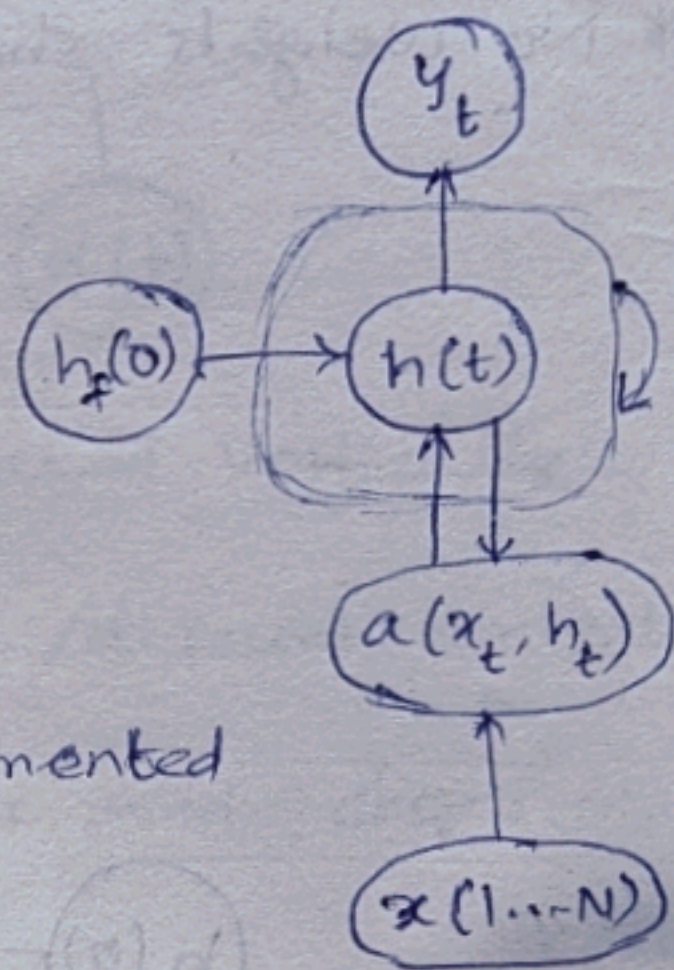
* For x_i , learn representations: key k_i and value v_i

→ key $k_i = g_k(x_i) = W_k x_i$ is a feature representing

the input - normalized to have unit norm.

→ value $v_i = g_v(x_i) = W_v x_i$ is a feature representing

the input of x_i useful for the current task.



→ For the state h_t , learn a representation:

query $q_t = g_v(h_{t-1}) = W_q h_{t-1}$ is a feature representing the current state.

* Compute the attention score as

$$a_t = s(q_t, k_i) = \sigma_N(q_t^T k_i) \rightarrow \text{inner product}$$

⇒ Projection gives similarity score; softmax gives a PMF of relevance probability of how relevant is the i^{th} input to the current state.

$$\text{Attention} = \text{softmax}(\text{Query}^T \text{Normalized-Key}) \text{Value}$$

* The weights W_k, W_v, W_q are shared for all i .

Multi Head Attention:-

* The inputs and states can have multiple relevant feature representations.

* For j^{th} feature of i^{th} input: $k_{ij}; v_{ij}; q_{ij} \{j=1, 2, \dots, L\}$

$$h_t = g(h_{t-1}, g_m(\sum_i a_{i1} g_{v1}(x_i), \dots, \sum_i a_{iL} g_{vL}(x_i)))$$

* This is cross-attention: computed b/w data and state.

- * **Self Attention**:- if query and key are computed for the same sequence. Basically, how one data sample is relevant to the other data samples in the same sequence. (Instead of computing query from the current state, the query is computed from the inputs directly). This is useful ~~for~~ to learn a context-aware representation for an input sample. (An input sample can represent mean different things depending on the surrounding samples).

Transformer Architecture:-

- * Fixed length input (context size): Input sequence is broken into tokens and mapped to embeddings.
- * Since attention is position invariant, a positional encoding vector is added to the input, $x_i = x_i + p_i$.
- * Self attention for inputs and outputs and cross attention ~~for~~ b/w encoder and decoder.
- * Layer norm and Skip connection for regularization.
- * Parallel training is possible; faster than RNN.
- * Masked self attention: mask out future values in the context window.
- * keys and values of past context windows are stored as KV cache.